

Voyager: An End-to-End Framework for Design-Space Exploration and Generation of DNN Accelerators

Kartik Prabhu
Stanford University
Stanford, USA
kprabhu7@stanford.edu

Jeffrey Yu
Stanford University
Stanford, USA
jeffreyy@stanford.edu

Xinyuan Allen Pan
Stanford University
Stanford, USA
allpan@stanford.edu

Zhouhua Xie
Stanford University
Stanford, USA
xzh015@stanford.edu

Abigail Aleshire
Stanford University
Stanford, USA
abbya@stanford.edu

Zihan Chen
Stanford University
Stanford, USA
zihan410@stanford.edu

Ammar Ali Ratnani
Stanford University
Stanford, USA
aratnani@stanford.edu

Priyanka Raina
Stanford University
Stanford, USA
praina@stanford.edu

Abstract

While deep neural networks (DNNs) have achieved state-of-the-art performance in fields from computer vision to natural language processing, efficiently running these computationally demanding models requires specialized hardware accelerators. However, designing these accelerators is a time-consuming, labor-intensive process that does not scale well across multiple design points. While prior efforts have sought to automate DNN accelerator generation, they typically offer limited parameterization, cannot produce high-performance, tapeout-ready designs, provide limited support for multiple datatypes and quantization schemes, and lack an integrated, end-to-end software compiler.

This work proposes Voyager, a high-level synthesis (HLS)-based framework for rapid design space exploration and generation of DNN accelerators. Voyager overcomes the limitations of prior work by offering extensive configurability across technology nodes, clock frequencies, and scales, with customizable parameters such as number of processing elements, on-chip buffer sizes, and external memory bandwidth. Voyager supports a much wider variety of datatypes and quantization schemes versus prior work, including both built-in arbitrary-length floating-point, posit and integer formats, as well as user-defined custom formats with both per-tensor scaling and microscaling quantization. Voyager’s PyTorch-based compiler efficiently maps neural networks end-to-end on the generated hardware, with support for quantization, operation fusion, and tiling.

We evaluate Voyager on state-of-the-art vision and language models. Voyager enables fast design-space exploration with full-dataset accuracy evaluation for different datatypes and quantization schemes. Generated designs achieve a high utilization across models and scales, up to 99.8%, and outperform prior generators with up to 61% lower latency and

56% lower area. Compared to hand-optimized accelerators, Voyager achieves comparable performance, while offering much greater automation in design and workload mapping.

1 Introduction

Deep neural networks (DNNs) are highly successful across many domains, but require substantial compute and memory resources. While many DNN accelerators have been proposed to address these challenges, designing efficient accelerators is an engineering-intensive process. As neural network models evolve rapidly, developing custom accelerators for each new type of model is increasingly impractical.

To address this, several DNN accelerator generators [4, 13, 15] have been proposed. While these frameworks help automate hardware generation, they often fall short in four key areas: (1) lack of support for comprehensive design space exploration, (2) inability to produce tapeout-ready systems, (3) lack of support for quantization schemes, such as microscaling, and (4) lack of a robust compiler to map high-level DNN models to the generated accelerators.

In this paper, we present Voyager, an end-to-end high-level synthesis (HLS)-based framework for design space exploration and generation of neural network accelerators. As shown in Figure 1, given architectural parameters such as data types, scaling factor type and granularity, compute parallelization, and buffer sizes, our framework produces performant, tapeout-ready register-transfer level (RTL) accelerator implementations. It also includes a co-designed compiler that efficiently maps PyTorch models to the generated hardware.

This paper makes the following contributions:

- We present an end-to-end high-level synthesis (HLS)-based framework for design-space exploration and generation of neural network accelerators. Our framework generates accelerators that are optimized for

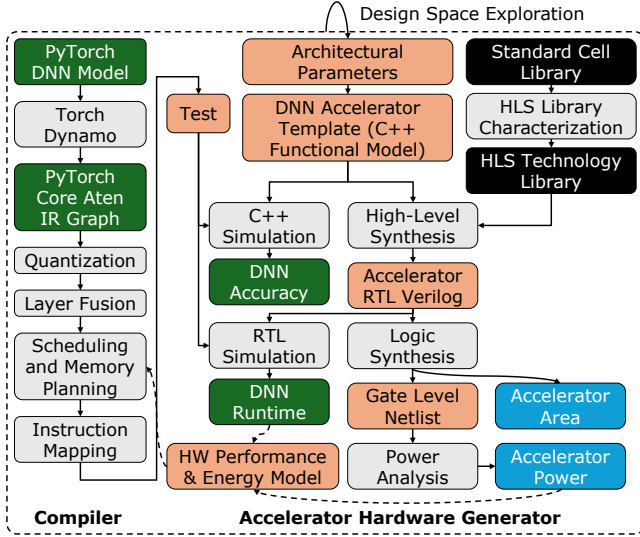


Figure 1. Voyager is an end-to-end framework for generating and evaluating neural network accelerators. It uses a high-level synthesis flow built around a templated accelerator design. Given architectural parameters, Voyager generates RTL implementations targeting a specified clock frequency and technology node. The C++ model enables fast evaluation of quantized DNN accuracy, while RTL simulation and synthesis provide runtime, energy, and area estimates.

a given set of machine learning models, technology node, and performance constraints. The design-space exploration tool supports accelerators with different numbers of processing elements, on-chip buffer sizes, and off-chip memory bandwidth, and generates designs with a highly-configurable dataflow, with programmable loop tiling, ordering, unrolling and fusion.

- Our generator supports a wider range of datatypes and quantization schemes compared to prior work. In addition to floating-point, posit, and integer datatypes with coarse-grained scaling factors supported by prior work, we additionally can generate accelerators with mixed-precision, multi-precision and custom user-defined types. Furthermore, we support any of these types with microscaling. Using our system, we can evaluate the costs and benefits of microscaling compared to coarser-grained scaling techniques. As opposed to prior work on microscaling that only evaluates accuracy using fake-quantization, our system leverages HLS to both generate hardware and provide fast quantized inference accuracy results that are bit-accurate with hardware. Particularly, our system accurately models the datatypes used for accumulation and non-linearities, unlike fake-quantization.
- Finally, we present a compiler that maps PyTorch models to any instance of the generated accelerator. Our

compiler supports quantization, hardware-aware operation fusion, and we integrate with our extended version of Interstellar [19] to find the optimal scheduling (loop tiling, ordering, unrolling and fusion) of a neural network on the accelerator.

We evaluate Voyager by generating accelerators that efficiently execute a wide variety of models used for image classification, language understanding and generation. Our results show that microscaled integers achieve inference accuracy that is within 1% of 32-bit floating point, but with up to 23% lower area than low-precision floating point formats. Voyager generated designs achieve up to 56% lower area and 61% lower runtime compared to prior generators, like Gemini and NVDLA at iso-hardware configuration. Compared to hand-optimized accelerators, we achieve up to 48% better performance, while providing greater design automation.

2 Related Work

Exploring the design space of DNN accelerators has been a focus of several prior works, which rely on architectural modeling. Tools such as Interstellar [19], Timeloop [11], MAESTRO [8], and ZigZag [10] calculate workload costs by generating mappings and evaluating them using an analytical cost model. Interstellar [19] estimates energy with a coarse-grained model of memory access and compute energy, using heuristics to prune the search space. Timeloop [11] adopts a similar approach, but supports exhaustive or random-sampling search. MAESTRO [8] focuses on scheduling search for a fixed dataflow. ZigZag [10] extends these works by offering greater flexibility in the architectural template and expanding the space of valid mappings. While useful for design-space exploration, these tools do not generate hardware.

Beyond modeling, several works generate synthesizable hardware for DNN accelerators, including Tandem Processor [5], DNNBuilder [20], MAERI [9], NVDLA [13], MAGNet [15], and Gemini [4]. These works differ in architecture, configurability, and software support.

Tandem Processor [5] targets non-GEMM operations and enables end-to-end execution of DNN models when combined with a GEMM unit. It uses software-managed scratchpads and a custom ISA to avoid the overheads of address calculation and loop control as in conventional processors. A custom compiler lowers ONNX graphs to the ISA.

DNNBuilder [20] generates designs for FPGAs with a pipelined architecture, mapping each pipeline stage to a neural network layer. The design is fixed to a single model, unlike other generators, including Voyager, that produce more general-purpose accelerators capable of supporting multiple DNN models.

MAERI [9] builds accelerators with a flexible interconnection fabric. The architecture consists of MAC units connected via a tree-based interconnection that supports a wide range of dataflows. MAERI can generate designs of various scales,

Feature	Tandem [5]	Builder [20]	MAERI [9]	NVDLA [13]	MAGNet [15]	Gemmini [4]	Voyager
Datatypes	Unspecified	Unspecified	Unspecified	Int/Float	Int	Int/Float	Int/Float/ Posit/Custom
Multi-Precision	✗	✗	✗	✗	✗	✗	✓
Microscaling	✗	✗	✗	✗	✗	✗	✓
Abstraction	RTL Verilog	RTL	RTL Verilog	RTL Verilog	HLS C++	RTL Chisel	HLS C++
SW Support	ONNX	Custom	Custom	Caffe	Custom	ONNX/C	PyTorch
SoC Integration	✗	✗	✗	✗	✗	✓	✓

Table 1. Comparison with prior DNN accelerator generators.

with configurable numbers of MAC units and SRAM sizes. However, it lacks a strong software stack.

NVDLA [13] supports a wide range of DNN operations via specialized blocks for convolution, activation functions, pooling, normalization, and reshaping. It offers configurability in datatypes, MAC array sizes, memory interface widths, and buffer sizes. Each specialized block can be independently sized and selectively included or excluded. Its software stack compiles Caffe models to hardware instructions.

MAGNet [15] uses an HLS template-based approach, with an array of PEs connected to a global buffer. Each PE integrates vector MAC units, local buffers, and post-processing units for ReLU, pooling, scaling, and quantization. It is configurable along several dimensions, including the number of PEs, buffer sizes, number of vector lanes, vector widths, and integer precision. However, MAGNet lacks a software stack and requires manual mapping of neural network layers.

Gemmini [4] is a Chisel-based generator of designs with PEs, scratchpad and accumulator memories, and additional units for pooling and activation functions. Gemmini can generate both systolic arrays and vector engines, with configurability over array sizes, buffer sizes, and the inclusion of optional blocks such as im2col, pooling, or transpose units. Gemmini also supports both arbitrary precision integer and floating-point data types. Its software stack adopts the ONNX format as the model specification.

In contrast to these works, Voyager generates accelerators that can accelerate neural networks end-to-end with far greater flexibility. Voyager supports a range of emerging datatypes, as well as advanced quantization strategies such as microscaling, which are increasingly critical for sub-8-bit precision. Prior generators also lack a robust software stack, requiring manual effort in converting models into custom formats for mapping them to hardware. In contrast, Voyager’s PyTorch-based compiler enables automatic quantization, fusion, and scheduling, enabling efficient execution on generated accelerators with minimal manual effort.

3 Voyager’s Accelerator Template

Voyager produces hardware instances from a flexible accelerator template, illustrated in Figure 2. The template has a matrix unit for convolution and general matrix multiplication

(GEMM), and a programmable vector unit for non-GEMM operations. Both units have custom instruction sets.

3.1 Matrix Unit

The matrix unit consists of a weight-stationary systolic array, where each processing element (PE) performs a multiply-accumulate (MAC) operation. Input channels are unrolled vertically, and output channels are unrolled horizontally. Inputs activations flow from left to right, while partial sums propagate from top to bottom. Weights are preloaded into PEs and remain stationary throughout a tile’s computation.

To maintain high utilization and avoid stalls from weight loading, each PE has triple-buffered registers interconnected by a column-wise shift-register chain. While one tile is being processed, weights for the next tile can be shifted in concurrently, enabling tile-level pipelining. Partial sums exiting the array are optionally combined with bias terms and written into a dedicated accumulation buffer.

The matrix unit supports runtime-configurable dataflow through programmable address generators that fetch tiles of inputs and weights from main memory and store them in independent double buffers, overlapping memory accesses with computation for higher throughput. These address generators have fine-grained control of loop ordering and bounds at both the L1 and L2 memory levels. Additionally, the input and weight controllers support in-place transpose and permute operations via register buffers, enabling fusion of transpose with GEMM operations.

3.2 Vector Unit

The N -wide multi-stage vector unit executes non-linear activations (e.g., ReLU, GeLU), element-wise operations (e.g., residual additional), complex functions such as softmax and layer normalization, and quantization/dequantization. It includes a multi-stage operation pipeline, a reduction unit for summation and max reduction, and an accumulator. Additionally, the unit includes three independent address generators that can access main memory.

To minimize memory accesses, outputs from the matrix unit feed directly into the vector unit, enabling fusion of matrix and vector operations. However, this tight coupling can stall the matrix unit if the vector unit is bottlenecked

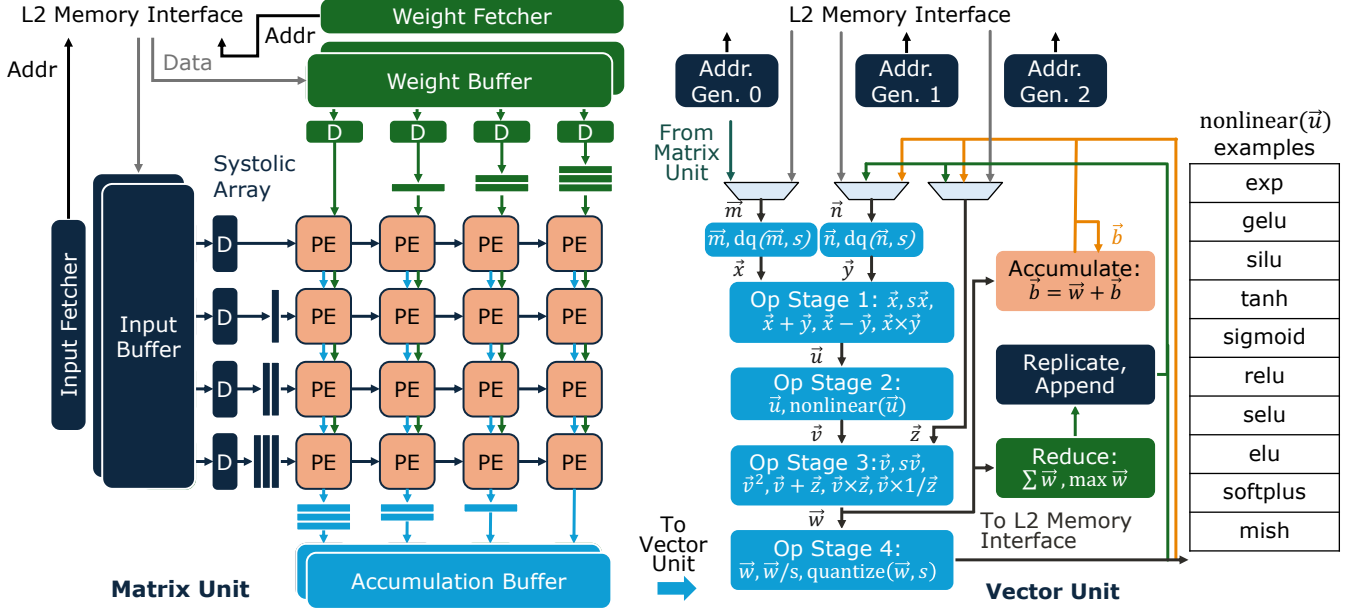


Figure 2. Voyager’s accelerator template has a matrix and a vector unit. Matrix unit has input and weight fetchers and buffers, an accumulation buffer, and a systolic array. Vector unit is a multi-stage pipeline for elementwise and reduction operations.

by memory access and exerts backpressure. To mitigate this, Voyager can generate designs with a double-buffered accumulation buffer: matrix unit outputs are instead stored in the buffer, and the vector unit independently reads from it while the matrix unit processes the next tile.

Within the operation pipeline, shown in Figure 2 on the right, each stage performs an element-wise operation or forwards data unmodified. To select operations for each stage, we first construct a dataflow graph of core operations commonly used in DNNs, such as layer normalization and softmax. By merging several dataflow graphs, we create a complete pipeline that maximizes the reuse of arithmetic operators without impacting latency. Outputs from the vector pipeline can be routed to memory, the accumulator, or the reduction unit. The accumulator and reducer outputs can be fed back into the pipeline as operands for subsequent operations, or written out to memory.

Nonlinear functions are supported through a configurable polynomial approximation block. Rather than using lookup tables or fixed polynomial circuits for each function, we create a single programmable unit. At compile time, each nonlinear function in the model is approximated with a free-knot spline of seven quadratic segments. Segment boundaries and coefficients are encoded into the instruction. At runtime, the unit selects the segment based on the input, and evaluates the polynomial, providing accurate, hardware-efficient support for nonlinearities.

To illustrate the vector unit’s operation, Figure 3 shows how softmax is mapped in three passes. First, the tensor is streamed from memory to the reduction unit to compute the

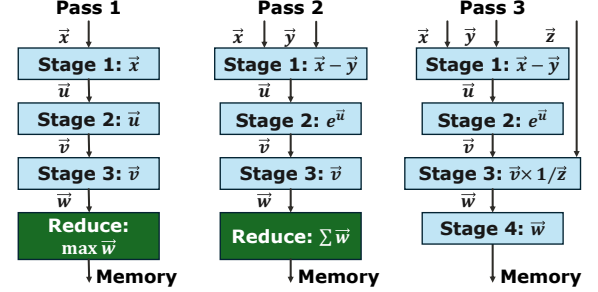


Figure 3. Three-pass mapping of softmax on vector unit.

maximum value across the reduction dimension, and then written back to memory. Next, both the original tensor and the maximum are read from memory. Each element is shifted by subtracting the maximum, exponentiated, and reduced to compute the normalization denominator, which is written back to memory. Finally, the original tensor and max and sum are read from memory, and the vector pipeline performs the max subtraction, exponentiation, and division by the sum to produce the final probabilities, which are written to memory.

3.3 User-Defined Extensions

Voyager supports user-defined compute blocks to specialize the accelerator for operations not efficiently handled by the matrix or vector units. For example, depthwise convolutions (DWC) exhibit limited data reuse compared to standard convolutions, making systolic array-based acceleration less effective. To address this, a user can integrate a dedicated

DWC unit, as shown in Figure 4. The block consists of input and weight fetchers, input line buffers, and a configurable number of MAC trees, each designed to reduce a 3×3 filter. For larger filter sizes, the compiler decomposes them into multiple 3×3 filters through slicing and padding, with the final accumulation performed in the vector unit. This keeps the hardware optimized for 3×3 filters, which dominate most models, while relying on the compiler to support other sizes.

By supporting such extensions, Voyager integrates specialized blocks seamlessly with its baseline architecture, broadening the range of workloads it can efficiently accelerate.

3.4 SoC Integration

Voyager-generated accelerators can be integrated into system-on-chip (SoC) platforms using the Chipyard [1] framework, as shown in Figure 5. Chipyard provides a complete infrastructure for building RISC-V SoCs, such as processor cores, TileLink-based interconnect generators, memory subsystems, and peripheral devices.

We leverage Chipyard’s support for custom accelerators by integrating Voyager-generated designs as memory-mapped I/O peripherals. The host CPU communicates with the accelerator through memory-mapped control registers, which are used to configure operations on the matrix and vector units. For memory accesses, the accelerator uses dedicated lightweight TileLink ports to connect to the system bus, enabling seamless access to on-chip L2 memory and off-chip DRAM.

This integration demonstrates that Voyager-generated designs are not only effective as standalone accelerators, but

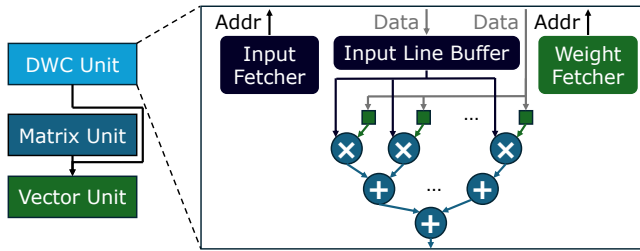


Figure 4. Example of a user-defined extension: a depthwise convolution unit with configurable MAC trees. Outputs flow directly into the vector unit for further processing.

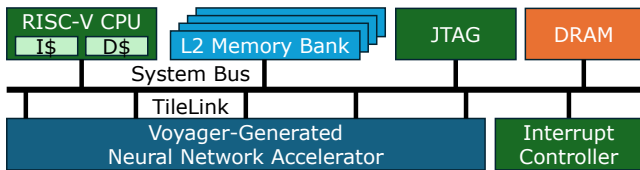


Figure 5. Voyager-generated accelerators can be integrated into an SoC using the Chipyard framework.

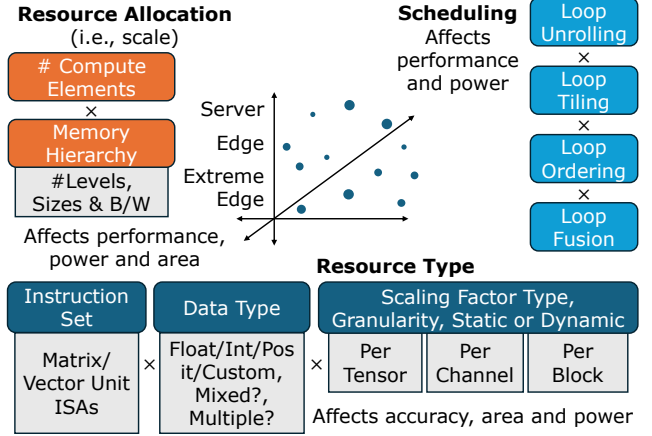


Figure 6. DNN accelerator design space.

also compatible with state-of-the-art SoC development flows, supporting both standalone and full-system evaluation.

4 Hardware Generator Parameterization

The Voyager generator exposes a large design space to the users, enabling them to generate designs that vary across three axes (Figure 6): resource allocation, resource type, and scheduling. Each axis captures a different aspect of the hardware (and software) design space and exposes tunable parameters that influence performance, power, area, and accuracy. To support this high degree of flexibility without sacrificing hardware efficiency, Voyager makes use of C++ metaprogramming features. Since HLS tools operate exclusively at compile-time, Voyager only utilizes the C++ features that enable compile-time specialization, particularly templates.

4.1 Resource Allocation: Compute/Memory Scaling

Voyager can generate designs of different sizes by configuring the number of compute elements and memory resources. For example, the dimensions of the systolic array are configurable to match the target workloads’ compute requirements, while the input, weight, and accumulation buffer sizes can be adjusted independently. Memory bandwidth to the main memory is also configurable. These parameters directly affect performance, area, and power and are chosen based on constraints such as latency targets or energy budgets. Larger systolic arrays reduce latency but incur higher area and power costs. Larger on-chip buffers enable greater data reuse and reduce main memory accesses, but increase buffer access energy and area [19]. These tradeoffs play a key role in design-space exploration and must be carefully balanced to meet system-level targets and constraints.

4.2 Resource Type: Datatypes and Quantization

Voyager can generate accelerators that support a variety of datatypes and quantization schemes, enabling users to

```

1  template<typename InputType, typename WeightType,
2      typename PsumType>
3  SC_MODULE(ProcessingElement) {
4      ...
5      void compute(InputType input, WeightType weight,
6          PsumType& psum){
7          psum = input.mac(weight, psum);
8      };
9  SC_MODULE(SystolicArray){
10      ...
11      ProcessingElement<int8, int8, int32> pe[16][16];
12  };

```

Listing 1. Generic processing element module.

generate accelerator variants tailored for specific classes of workloads. For instance, users can generate a standard BFloat16 accelerator, an INT8 accelerator with per-channel scaling, or an FP6 accelerator with microscaling.

Datatype Parameterization. Voyager accomplishes hardware datatype parameterization through C++ templates. Templates in C++ enable generic programming, allowing developers to define functions and classes parameterized on type. During compilation, the compiler performs type substitution and generates specialized versions of these generic functions and classes with concrete types.

As shown in Listing 1, the ProcessingElement module is the compute element of the systolic array, and performs a MAC operation on three operands. This module is templated on three datatypes: InputType for inputs, WeightType for weights, and PsumType for partial sums. The SystolicArray module then instantiates an array of ProcessingElements, with concrete types for inputs, weights, and partial sums.

Voyager provides three datatype families by default: integer, floating-point, and posit [6], each implementing arithmetic and logical function according to its own semantics. These classes are also templated on configuration parameters, such as bit width, exponent size, or signedness (Listing 2). Each type implements a mac function, enabling seamless type substitution during compilation with C++ templates.

In addition to the built-in types, Voyager supports extensibility through user-defined datatypes. To add a new type, users simply define a class that implements the expected interface, including the mac function and other arithmetic and logical operations. This mechanism is flexible enough to support configurations such as vectorized PEs (Listing 3) and unconventional formats like a codebook-based NormalFloat4 [3] (Listing 4).

Voyager also supports multi-precision designs, where the hardware can operate on different datatypes that are selectable at run-time. For example, some layers in a model are more sensitive to quantization errors and may require

```

1  template<int Width, bool Signed>
2  class Int {
3      ac_int<Width, Signed> value;
4      Int mac(Int weight, Int psum) {
5          return value * weight.value + psum.value;
6      }
7  };
8  template<int MantissaBits, int ExponentBits>
9  class FP {
10     ac_std_float<MantissaBits, ExponentBits> value;
11     FP mac(FP weight, FP psum) {
12         return value.fma(weight, psum);
13     }
14 };
15 template<int Width, int EsBits>
16 class Posit {
17     ac_int<Width, false> bits;
18     Posit mac(Posit weight, Posit psum) {
19         ...
20     }
21 };

```

Listing 2. Datatype class implementations.

```

1  template<typename T, int Width>
2  class Vector {
3      T data[Width];
4      Vector mac(Vector weight, Vector psum) {
5          for(int i = 0; i < Width; i++) {
6              data[i].mac(weight[i], psum[i]);
7          }
8      };
9  };

```

Listing 3. Voyager supports vectorized PEs.

```

1  class NormalFloat4 {
2      ac_int<4, false> bits;
3      static const FP codebook[16];
4      NormalFloat4 mac(NormalFloat4 weight, NormalFloat4
5          psum) {
6          for(int i = 0; i < Width; i++) {
7              codebook[bits].mac(codebook[weight.bits],
8                  codebook[psum.bits]);
9          }
10     };
11 };

```

Listing 4. Voyager supports novel datatypes.

higher precision, while others can tolerate lower precision to improve efficiency.

To enable this flexibility, Voyager uses variadic C++ templates and partial template specialization to generate hardware modules that can seamlessly support multiple datatypes within the same design. For example, the InputController fetches tiles of inputs from L2 memory. In order to support

```

1  template <typename InputTypes>
2  struct InputController;
3
4  template <typename... InputTypes>
5  struct InputController<std::tuple<InputTypes
6      ...>>{...};
7
8  // InputController instantiated with a single type
9  InputController<std::tuple<FP8>> inputController;
10 // or multiple types
11 InputController<std::tuple<FP16,FP8,FP4>>
    inputController;

```

Listing 5. The InputController template supports multiple datatypes through variadic templates using tuples.

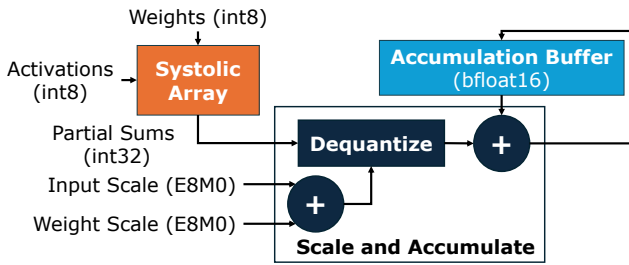


Figure 7. Voyager can generate accelerators with microscaling by introducing dequantization logic in the matrix unit.

fetching multiple datatypes, this module is instantiated with tuples of supported types, as shown in Listing 5. This approach generates datatype-specific fetch logic that can be selectively used at run-time. The compiler encodes the precision required for each layer into the instruction, enabling the hardware to flexibly support multiple precisions.

Quantization Scheme Parameterization. To support a wide range of accuracy-efficiency tradeoffs, Voyager can generate accelerators with various quantization schemes, including per-tensor and per-channel quantization. To support these schemes, Voyager-generated accelerators consist of a quantization stage at the output of the vector unit as well as dequantization hardware within the vector unit’s fetcher, allowing for fusion of quantization and dequantization with the previous operation.

Voyager can also generate accelerators that support **microscaling** [12], a quantization scheme that applies block-wise scaling along the reduction dimension to improve accuracy in low-precision GEMM operations. Supporting this scheme requires specialized hardware to both efficiently perform the GEMM operation and to generate scaling factors per output block. As shown in Figure 7, Voyager inserts additional dequantization logic within the matrix unit. Inputs and weights are stored in a low-precision integer format, such as

```

// Sub-optimal C++ coding style for HLS
for(int ic = 0; ic < IC_loop_bound; ic++){
    for(int oy = 0; oy < OY_loop_bound; oy++){
        for(int ox = 0; ox < OX_loop_bound; ox++){
            int address = oy * OX_loop_bound * IC_loop_bound
                + ox * IC_loop_bound + ic;

            ...
        }
    }
}

// Coding style for improved results
int IC_last = IC_loop_bound - 1;
int OY_last = OY_loop_bound - 1;
int OX_last = OX_loop_bound - 1;
int OY_stride = OX_loop_bound * IC_loop_bound;
for(int ic = 0; ic < IC_last; ic++){
    for(int oy = 0; oy < OY_last; oy++){
        for(int ox = 0; ox < OX_last; ox++){
            int address = oy * OY_stride
                + ox * IC_loop_bound + ic;

            ...
            if(ox == OX_last) { break; }
            if(oy == OY_last) { break; }
        }
        if(ic == IC_last) { break; }
    }
}

```

Figure 8. Coding style has a large impact on HLS-generated RTL. We restructure loops from a standard nested loop (top) to one with constant precomputation and breaks (bottom).

INT8, and each have an associated scale factor represented in E8M0 format. The systolic array accumulates in a higher precision integer format, such as INT32, and dequantizes the partial sum to a floating-point format, such as BF16, using the product of the input and weight scales. This scaled partial sum is then accumulated with a previous partial sum and stored in the accumulation buffer.

4.3 Scheduling: Loop Transformations

While previous axes define what hardware is generated, the scheduling axis determines how computation is mapped to that hardware. Voyager generates accelerators with runtime-configurable loop counters and address generators that control data movement and memory access patterns. These controllers allow the accelerator to support a wide range of loop ordering and tiling strategies. Scheduling directly impacts data reuse, which in turn affects energy efficiency and latency. The compiler’s strategy for schedule selection and lowering is described in Section 5.5.

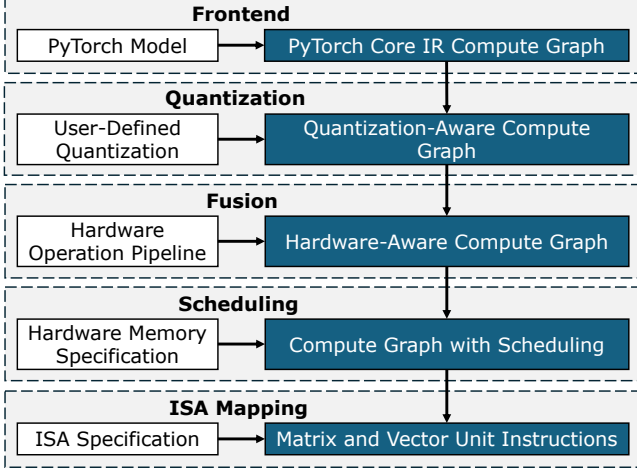


Figure 9. The Voyager compiler lowers PyTorch models to hardware instructions through quantization, hardware-specific optimizations such as fusion and tiling, and generation of matrix/vector instructions for the accelerator ISA.

4.4 HLS C++ Coding Style Optimizations

The C++ coding style used in the generator has a significant impact on the quality of the RTL generated from HLS, affecting area, timing, and HLS tool’s runtime. Voyager’s address generators are implemented using nested for loops, with address computation logic in the loop body. To improve HLS results, we perform two optimizations shown in Figure 8:

- (1) **Pre-computation of constants:** Any values that can be computed outside the loop nests, such as loop bounds or loop strides, are moved out of the body. This reduces the dependency chain of operations within the pipeline, improving timing. Additionally, this reduces the number of arithmetic operations that the HLS tool must analyze while scheduling.
- (2) **Flattened loop control:** We restructure the loops as infinite loops with explicit break conditions in the body. This avoids the need for a final extra iteration introduced by the HLS tool when evaluating loop bounds in the loop header. This speeds up the inner loop which has a significant impact, since there are many levels of nested loops.

5 Voyager’s Compiler

To efficiently deploy models onto Voyager-generated accelerators, we co-designed a compiler with the hardware template. Like many DNN accelerators, Voyager uses a domain-specific instruction set, requiring a custom compiler to transform high-level models into optimized hardware instructions. The compiler consumes PyTorch models and lowers them through stages of quantization, hardware-aware optimizations, and scheduling, as shown in Figure 9.

To provide a visual understanding of Voyager’s software compilation flow, we show how a simple example PyTorch

Example PyTorch Model

```

class Example(nn.Module):
    def __init__(self):
        super(Example, self).__init__()
        self.conv = nn.Conv2d(
            in_channels=32,
            out_channels=64,
            kernel_size=3,
            stride=1)
        self.relu = nn.ReLU()
    def forward(self, x):
        x = self.conv(x)
        x = self.relu(x)
        return x
  
```

Static Compute Graph

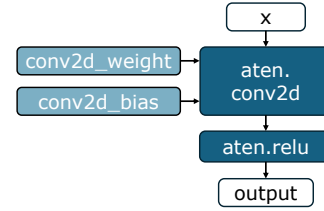


Figure 10. The frontend extracts a static compute graph from a PyTorch model using PyTorch 2 Export. The original model (top) is lowered to an intermediate representation consisting of ATen operators (bottom), the input to later compiler stages.

model, with a convolution layer followed by a ReLU, gets lowered to the hardware accelerator’s ISA.

5.1 Frontend

Our compiler’s frontend converts PyTorch models into an intermediate representation (IR) that bridges the gap between the high-level model and the accelerator’s instruction set architecture (ISA). We use PyTorch 2 Export (PT2E) [2] to extract a static compute graph, as shown in Figure 10. We adopt PyTorch as the frontend due to its widespread use and ecosystem, including torchvision [14] and Hugging Face Transformers [18]. Users can import existing models into the Voyager toolchain with minimal modification.

5.2 Quantization

While most ML frameworks natively support quantization, including post-training quantization and quantization-aware training, they support only a limited set of datatypes and quantization schemes. They do not offer built-in support for state-of-the-art quantization techniques. To address this, our compiler incorporates a custom quantization framework, where users have fine-grained control over quantization behavior for weights, activations, and biases, as shown in

Quantization Specification

```
quantizer = get_default_quantizer(
    inputs="int8,qs=per_tensor",
    weight="int8,qs=per_tensor",
    bias="int32",
)
model = pt2e(model, quantizer)
```

Quantization-Aware Compute Graph

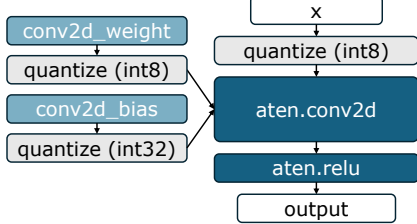


Figure 11. Voyager inserts quantization operators into the compute graph to emulate quantized execution. The user specifies a quantization configuration (**top**), which is applied to the model and results in a quantization-aware compute graph (**bottom**).

Figure 11. The quantizer annotates graph nodes with quantization intent and inserts fake quantization operators to emulate quantized behavior in the floating-point model.

Data Types. Voyager extends datatype support to arbitrary-bitwidth integers, floating-point, posit, and NormalFloat. Users can define custom types through a fake quantization function that maps BFloat16 values to the custom type.

Quantization Schemes. We support a wide range of quantization schemes. While PyTorch provides per-tensor and per-channel scaling, which are commonly used for int8 and FP8 quantization, these methods fail to maintain accuracy as precision drops below 8 bits. Microscaling, also known as block-wise quantization, has shown notable accuracy improvements for sub-8-bit quantization. Smaller block sizes generally offer higher accuracy but at the cost of increased bits-per-parameter and computation. To our knowledge, ours is the first compiler to support microscaling with fully configurable block sizes and data types.

Quantization Granularity. We provide fine-grained quantization control, allowing different datatypes and quantization schemes per layer based on hardware constraints and performance goals. This is essential for mixed-precision quantization, where less sensitive layers can use lower precision to reduce model size.

5.3 Fusion

Voyager performs three types of fusion: quantization fusion, operation fusion, and reshape fusion.

Quantization Fusion. After quantization, our compiler replaces fake quantization modules with hardware-executable

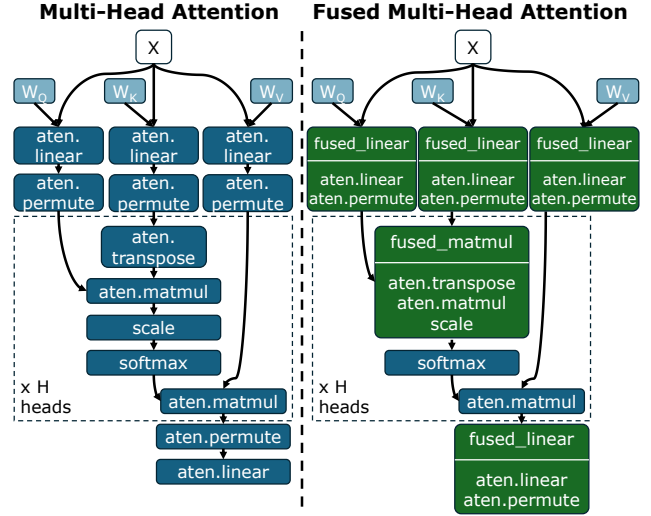


Figure 12. Reshape and matmul fusion in self-attention.

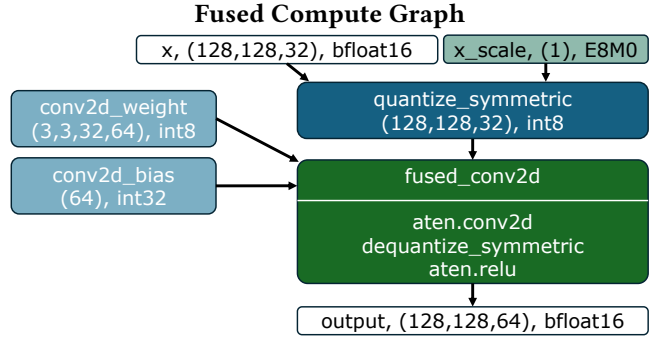


Figure 13. The compiler performs fusion of quantization and operation nodes. In this example, it replaces convolution-dequantization-ReLU with a single fused operation.

quantize and dequantize operators. Quantize scales a floating-point tensor and then casts it to the target data type, while dequantize converts a quantized tensor back to floating-point by dividing it by the corresponding scale. Scale factors are computed from tensor statistics observed during calibration runs over sample data. Voyager supports both power-of-two scaling for better hardware efficiency and general floating-point scaling for higher accuracy. When direct quantization is used (e.g., scale factor of 1), redundant dequantize operators are removed. To enable integer-only GEMM, dequantization is deferred until after GEMMs, keeping computation in integer format for improved energy efficiency.

Operation Fusion. The compiler combines multiple smaller operations into a single kernel, avoiding intermediate writes to memory and increasing hardware utilization. We support fusing matrix operations with subsequent element-wise vector operations, as well as across multiple vector operations.

Fusion is guided by the vector pipeline structure: the compiler replaces sequential patterns that match the supported hardware pipeline and replaces them with fused instructions. This transformation is illustrated in Figure 13, where a sequence of convolution, dequantization, and ReLU activation is replaced with a single fused operator.

Reshape Fusion. The compiler also fuses reshape and dequantize operations with their consumers. The vector unit’s address generators can directly perform reshaping operations such as transpositions and axis permutations during memory access, eliminating intermediate tensors and reducing both latency and memory overhead. This is especially beneficial for transformers, where multi-head attention involves several reshapes: query, key, and value projections are permuted to separate the head dimension, the key is transposed, and outputs of the attention heads are recombined by another permutation. As shown in Figure 12, the compiler fuses linear projections with permutation and transpose, producing a self-attention graph with fewer memory accesses and lower latency.

5.4 Accelerator IR Generation

To enable downstream optimizations and hardware mapping, the Voyager compiler generates a hardware-oriented IR tailored for Voyager-generated accelerators. As illustrated in Figure 14, The IR captures both the high-level operations and low-level details, such as datatypes, tensor shapes, and memory addresses. The core unit is a Tensor object, mirroring a PyTorch tensor but enriched with hardware-relevant metadata, such as shape, datatype, memory address, and an optional permutation field to support implicit reshaping (e.g., transpositions).

PyTorch operations are categorized into four main groups: (1) **matrix operations** including convolution and general matrix multiplication (GEMM), (2) **elementwise operations** such as ReLU, GeLU, and residual additions, (3) **complex reductions** like LayerNorm and softmax, and (4) **reshape operations** like permute, view, and transpose. Each graph node maps to an IR block of type (1), (3), or (4). If a node represents a fused operation, its element-wise functions (category 2) are appended as a post-processing list in the same IR block. For example, a convolution followed by residual addition and ReLU is represented as a matrix operation with fused element-wise transformations.

5.5 Scheduling

Mapping DNNs onto hardware involves making scheduling decisions such as loop tiling, loop ordering, and loop fusion. The space of possible schedules is large, with each producing distinct data reuse patterns across memory levels. Optimizing one type of data reuse reduces opportunities for another. Voyager addresses this by extending Interstellar [19] to efficiently schedule DNN layers onto the generated hardware.

```
name: "conv2d_fused"
op_list: [{
  name: "conv2d"
  target: "conv2d"
  input:
    node: "quantize_symmetric"
    shape: [128,128,32]
    dtype: "int8"
    address: 18688
  weight:
    node: "conv2d_weight"
    shape: [3,3,32,64]
    dtype: "int8"
    address: 0
  bias:
    node: "conv2d_bias"
    shape: [64]
    dtype: "int32"
    address: 18432
}],
{
  name: "dequantize_default"
  target: "dequantize"
  dtype: "int24"
  input:
    node: "conv2d"
    shape: [128,128,64]
    dtype: "int32"
  scale:
    node: "conv2d_scale"
    shape: [1]
    dtype: "bfloat16"
},
{
  name: "relu"
  target: "relu"
  input:
    node: "dequantize_default"
    shape: [128,128,64]
    dtype: "bfloat16"
}]
output:
  node: "conv2d_fused"
  shape: [128,128,64]
  dtype: "bfloat16"
  address: 542976
```

Figure 14. The compiler generates an intermediate representation (IR) consisting of operations, tensor metadata (shape, datatype), and memory addresses.

```

# L2 level
for oy1 in range(16):
    for ox1 in range(4):
        for oc2 in range(2):
            # Accumulation buffer level
            for ic1 in range(2):
                for oc1 in range(2):
                    for fx in range(3):
                        for fy in range(3):
                            for oy0 in range(8):
                                for ox0 in range(32):
                                    # Systolic array level
                                    parallel_for ic0 in range(16):
                                        parallel_for oc0 in range(16):

                                            # Compute shape
                                            input_height = 16*8
                                            input_width = 4*32

                                            # Compute indices
                                            oy = oy1 * 8 + oy0
                                            ox = ox1 * 32 + ox0
                                            oc = oc2 * 2 * 16 + oc1 * 16 + oc0
                                            ic = ic1 * 16 + ic0

                                            # Adjust for padding
                                            in_y = oy - 1
                                            in_x = ox - 1

                                            # Add bias
                                            if ic == 0 and fx == 0 and fy == 0:
                                                output[oy,ox,oc] = bias[oc]

                                            # Perform MAC
                                            if (0 <= in_y < input_height) and
                                                (0 <= in_x < input_width):
                                                output[oy,ox,oc] += input[in_y,in_x,ic]
                                                    * weight[fy,fx,ic,oc]

                                            # Apply ReLU activation
                                            if ic==31 and fy==2 and fx==2:
                                                output[oy,ox,oc] = max(output[oy,ox,oc], 0)

```

Figure 15. An example loop schedule generated using an extended version of Interstellar.

Interstellar uses a coarse-grained analytical cost model that estimates memory energy by counting accesses at each level of the memory hierarchy and applying per-access energy weights from a hardware-specific lookup table. A heuristic search then explores the schedule space efficiently.

We extend Interstellar in two ways: (1) we constrain the schedule space to only the schedules feasible on Voyager-generated accelerators, and (2) add a performance model that accounts for runtime based on hardware characteristics.

Voyager’s performance model breaks down execution time into two components: **(1) L1 tile compute time:** This is based on the tile size and the number of compute elements in the systolic array. Before computation, weights must be loaded into the systolic array, requiring one cycle per row in the array. To hide this latency, the weights within PEs are double-buffered, so weight loading for the next tile is overlapped with computation of the current tile. Thus, compute time per tile is the maximum of (a) the time to perform the computation, set by the tile size and PE latency, and (b) weight loading time. The total latency for an L1 tile equals this per-tile latency multiplied by the number of systolic array tiles. **(2) L1 tile memory access time:** This is how long it takes to transfer tiled input and weight tensors into the L1 buffers, determined by the memory bandwidths.

The total runtime is the maximum of the L1 compute and memory access time, multiplied by the number of tiles in L2 memory. If the design lacks a double-buffered accumulation buffer, the model also accounts for vector unit backpressure by including its compute time in the maximum. For deeper memory hierarchies, this process is repeated at each level. By incorporating these hardware-specific effects, our performance model provides fast and accurate runtime predictions.

An example of a generated schedule is shown in Figure 15, which illustrates a multi-level loop nest for a convolution.

5.6 Accelerator Instruction Generation

Using the accelerator IR and the schedule from Interstellar, the compiler maps high-level operations onto Voyager hardware instructions. Matrix unit instructions encode loop configuration fields, such as bounds, orderings, and strides. Interstellar’s loop nests map directly to this structure, allowing schedules to be translated into address generator configurations with minimal transformation. Vector unit instructions also include address generator configurations, but extend to specify the vector pipeline configuration. This includes selection of functional units for each stage, as well as settings for the accumulation and reduction engines, and quantization/dequantization. Together, these instructions enable the compiler to efficiently map diverse operations, including fused patterns, onto the accelerator.

Figure 16 shows a subset of the matrix unit instruction. The base address fields specify the memory addresses for tensors in main memory. The loops field encodes loop bounds at two levels of hierarchy, as specified by the schedule generated by Interstellar. The *_loop_index fields map each dimension to its respective index in the loops field.

```

struct MatrixUnitInst {
    uint64_t input_base_address = 18688;
    uint64_t weight_base_address = 0;
    uint64_t bias_base_address = 18432;

    uint16_t loops[2][6] =
    {
        {16,4,2,1}, // L2 level
        {2,2,3,3,8,32} // accum buffer level
    };
    uint3_t x_loop_index[2] = {1,5};
    uint3_t y_loop_index[2] = {0,4};
    uint3_t reduction_loop_index[2] = {3,0};
    uint3_t output_channel_loop_index[2] =
        {2,1};
    uint3_t fx_index = 2;
    uint3_t fy_index = 3;
    ...
};

```

Figure 16. A subset of a matrix unit instruction used to configure memory addresses, loop bounds, and loop orderings.

		Runtime	Speedup
Fusion	Baseline	2.67 ms	-
	+ Permute & Transpose	2.65 ms	0.75%
	+ Residual	2.61 ms	2.30%
	+ NoNorm	2.37 ms	12.66%
Model	Energy-Only	2.44 ms	-
	Coarse-Grained Perf. Model	2.44 ms	-
	Fine-Grained Perf. Model	2.37 ms	2.86%
	Without Double Buffer	2.37 ms	-
	With Double Buffer	2.29 ms	3.37%

Table 2. Runtime improvement from operator fusion, fine-grained performance modeling and double buffering.

6 Results

We evaluate Voyager across multiple design points using DNNs models from different application domains. Our evaluation flow includes high-level synthesis using Catapult HLS to generate RTL Verilog, cycle-accurate simulations using Synopsys VCS, and logic synthesis using Synopsys Design Compiler. MAC utilization is defined as the ideal cycles divided by the achieved cycles, where the ideal cycles is the total number of MAC operations in a layer divided by the number of hardware MAC units. In Section 6.1, we analyze the impact of key design decisions within Voyager. Section 6.2 shows how we can use Voyager for design space exploration. Finally, Sections 6.3 and 6.4 compare Voyager against state-of-the-art hardware generators and hand-optimized designs.

6.1 Optimizations within Voyager

We evaluate the impact of optimizations in Voyager using a 32×32 E4M3 design in TSMC 16 nm on MobileBERT-TINY.

Coding Style Optimizations. With the unoptimized coding style, the design is limited to a maximum frequency of 400 MHz, with an area of 1.16 mm². In contrast, the optimized coding style substantially improves timing closure, enabling a maximum frequency of up to 1 GHz, with an area of 1.41 mm². When normalized to the same frequency (400 MHz), the optimized design has an area of 1.18 mm², only a slight area increase of 1.7% versus the unoptimized version.

Operator Fusion. Voyager can fuse operators, which reduces memory accesses and improves latency. Table 2 shows the impact of various degrees of fusion on latency, with a final runtime improvement of 12.66%.

Scheduling. Table 2 compares runtime using three scheduling strategies: an energy-only heuristic, a coarse-grained performance model, and our fine-grained performance model, which achieves up to 2.9% lower runtime, showing the value of accurate performance modeling in schedule selection.

Double Buffering. The tight coupling between the matrix and vector units can result in slowdowns in situations where the vector unit is bottlenecked by memory accesses. To mitigate this, Voyager supports an optional double-buffered accumulation buffer, decoupling its output from the vector unit’s input, leading to a 3% reduction in runtime (Table 2).

6.2 Design Space Exploration with Voyager

To show Voyager’s design space exploration capability we present how we can use it to explore (1) different datatypes and quantization schemes (resource type) and (2) accelerator sizes (resource allocation) on various DNNs. For these studies, we use the optimal schedules from our compiler.

Exploring Datatypes and Quantization Schemes. Using the bit-accurate C++ model, Voyager allows running inference on entire datasets to evaluate hardware-level accuracy on different DNNs for different datatypes and quantizations. Table 3 shows the metrics for vision and NLP models quantized to different types (FP32, BF16, E4M3, Posit8, INT8 and MXINT8). We use pre-trained models from torchvision [14], timm [17], and transformers [18] libraries. These results show that microscaling (MXINT8) enables high-accuracy integer computation, achieving accuracy on par with floating-point formats. Voyager makes it possible to evaluate this across diverse models. However, using microscaling can come at an area cost. Voyager allows us to evaluate this as well. Table 5 reports the area of Voyager-generated accelerators in TSMC 16 nm at 1 GHz clock frequency. MXINT8 requires 43% more area for addressing and control than INT8 (for a 32×32 design) due to the added support for input and weight scale factors. However, its overall area is still 5.64% lower than INT8 due to a smaller accumulation buffer and

Type	Model	Task (Dataset)	Size	Accuracy					
				FP32	BF16	E4M3	Posit8	INT8	MXINT8
Vision	ResNet-18	Image Classification (ImageNet)	11.7M	71.5%	70.0%	69.2%	69.1%	71.5%	70.7%
	ResNet-50	Image Classification (ImageNet)	25.6M	80.4%	81.1%	78.8%	79.4%	78.7%	79.8%
	ViT (Base)	Image Classification (ImageNet)	86M	84.1%	84.3%	83.8%	84.0%	75.4%	84.0%
	MobileNetV2	Image Classification (ImageNet)	3.5M	71.1%	70.1%	62.5%	63.8%	66.0%	71.3%
NLP	MobileBERT-TINY	Sentiment Classification (SST-2)	15M	90.8%	90.7%	90.6%	90.4%	90.4%	91.1%
	BERT (Base)	Sentiment Classification (SST-2)	110M	93.2%	93.0%	93.1%	92.8%	92.4%	93.1%

Table 3. Accuracy of vision and NLP models across different data types. NLP models are run with a sequence length of 128.

Model	Metric	Array Size and Total SRAM Capacity						
		8x8 (48 KB)	8x16 (80 KB)	16x16 (96 KB)	16x32 (160 KB)	32x32 (192 KB)	32x64 (320 KB)	64x64 (384 KB)
ResNet-18	Runtime (cycles)	31.76M	15.88M	7.90M	3.96M	2.07M	1.04M	660K
	MAC Utilization (%)	90.9	90.9	93.0	92.9	91.9	91.4	76.8
ResNet-50	Runtime (cycles)	67.70M	33.86M	17.00M	8.51M	4.40M	2.21M	1.30M
	MAC Utilization (%)	95.4	95.4	96.2	96.1	94.9	94.5	82.1
ViT (Base)	Runtime (cycles)	326.19M	163.14M	84.37M	42.24M	22.74M	12.44M	7.53M
	MAC Utilization (%)	99.4	99.4	99.3	99.2	98.0	96.5	88.0
MobileNetV2	Runtime (cycles)	7.373M	3.760M	2.010M	1.099M	659K	534K	407K
	MAC Utilization (%)	86.7	84.9	83.7	79.3	78.0	58.6	52.5
MobileBERT-TINY Sequence Length = 128	Runtime (cycles)	27.91M	14.00M	7.71M	3.91M	2.37M	1.28M	916K
	MAC Utilization (%)	98.3	98.0	95.1	93.8	86.2	82.8	67.3
BERT (Base) Sequence Length = 128	Runtime (cycles)	179.17M	89.63M	46.07M	23.09M	12.24M	6.48M	3.80M
	MAC Utilization (%)	99.8	99.8	99.4	99.2	97.9	96.2	90.7
LLaMA 3.2 1B Prefill Sequence Length = 512	Runtime (cycles)	–	–	–	–	688.0M	–	–
	MAC Utilization (%)	–	–	–	–	99.8	–	–

Table 4. Runtime and utilization across E4M3-based designs at different scales.

vector unit, and 11.6% lower than E4M3 due to a smaller systolic array.

Exploring Accelerator Sizes. Voyager designs scale effectively across systolic array sizes (Table 4). Runtimes decrease nearly linearly with array size, maintaining a high hardware utilization.

6.3 Comparison with Prior Generators

Table 6 compares Voyager designs against those from two other generators, Gemmini and NVDLA. For a fair comparison, we match the technology node, datatype, number of MACs and frequency to the design points reported in prior work. While Gemmini achieves a 28% lower area, Voyager is significantly more performant, with a 61% lower runtime. Compared to NVDLA, Voyager designs are 35.91% to 56.48% smaller and achieve 20.84% to 53.15% lower runtime.

6.4 Comparison with Hand-Optimized Designs

We compare Voyager with two prior hand-optimized accelerators: Simba [16, 21, 22], a CNN accelerator, and a transformer accelerator with vector-scaled quantization (VSQ) [7]. We match design parameters as much as possible. Because

the hand-optimized designs report post-layout results, we scale up the area of Voyager designs by 1.25 $\times$ to account for place-and-route overhead.

To compare with Simba, we generate a design in the same TSMC 16nm process with matching MACs, datatype, and buffer sizes (Table 7). While Simba reports results at a higher clock frequency, this is due to a much higher voltage than our characterized libraries, making a direct runtime comparison difficult. Comparing latency in cycles, Voyager greatly outperforms Simba, with a ResNet-50 latency of 4.4 million cycles compared to Simba’s 8.3 million. Voyager’s utilization is 95% vs. Simba’s 48%. Even with the lower frequency, the Voyager outperforms Simba by 5% on absolute runtime.

For the transformer VSQ design, we generate a design in TSMC 7 nm (due to lack of access to 5 nm) with matching number of MACs, as shown in Table 7. Despite the older technology node, Voyager can operate at the same clock frequency. On BERT-Base, Voyager achieves a runtime of 13.4 ms, compared to VSQ’s 11.4 ms. However, unlike Voyager, VSQ does not accelerate the entire model, as it lacks support for reshaping, transposition, residual addition, and layer normalization. In terms of MAC utilization, both designs achieve

Size	Type	Area (mm ²)				
		Sys. Array	SRAM	Addr. & Ctrl.	Vector Unit	Total
8x8	E4M3	0.048	0.085	0.062	0.142	0.336
	Posit8	0.052	0.085	0.064	0.145	0.346
	INT8	0.027	0.111	0.064	0.148	0.350
	MXINT8	0.031	0.095	0.095	0.135	0.355
16x16	E4M3	0.192	0.109	0.096	0.237	0.633
	Posit8	0.210	0.109	0.101	0.242	0.661
	INT8	0.109	0.211	0.098	0.250	0.667
	MXINT8	0.123	0.123	0.161	0.224	0.632
32x32	E4M3	0.769	0.217	0.199	0.442	1.627
	Posit8	0.846	0.217	0.215	0.450	1.728
	INT8	0.440	0.422	0.198	0.465	1.525
	MXINT8	0.501	0.243	0.283	0.412	1.438
64x64	E4M3	3.104	0.434	0.453	0.825	4.817
	Posit8	3.399	0.434	0.506	0.836	5.175
	INT8	1.783	0.844	0.432	0.869	3.928
	MXINT8	2.021	0.484	0.565	0.752	3.821

Table 5. Post-synthesis area of different accelerator configurations in 16 nm targeting a clock frequency of 1 GHz.

# MACs	Area (μm^2)		ResNet-50 Time (ms)	
	Gemmini	Voyager	Gemmini	Voyager
256	116,000	160,314	43.58	17.00
	NVDLA	Voyager	NVDLA	Voyager
2048	3,300,000	2,114,852	3.72	2.21
1024	1,800,000	1,102,659	6.54	4.40
512	1,400,000	852,504	10.75	8.51
256	1,000,000	456,468	21.74	17.00
128	840,000	394,511	50.00	33.86
64	550,000	239,519	136.99	67.70
32	510,000	221,943	277.78	130.13

Table 6. Comparison of Voyager with Gemmini in Intel 16 and NVDLA in TSMC 16. All designs use INT8 compute at 1 GHz. Areas are standard cell area (compute array only for Gemmini comparison, full design for NVDLA comparison).

98%. These comparisons show that Voyager can match or outperform hand-optimized designs in performance and utilization, while providing greater design automation.

7 Conclusion

Voyager is an end-to-end framework for generating DNN accelerators. Its HLS based accelerator generator provides extensive configurability in datatypes and quantization schemes. In addition, its PyTorch-based compiler maps models efficiently to generated hardware, supporting quantization, operation fusion, and tiling. Together, these components enable

Metric	Simba	Voyager	VSQ	Voyager
# MACs	1024	1024	1024	1024
Datatype	INT8	INT8	INT4	INT4
Technology	16nm	16nm	5nm	7nm
Frequency (GHz)	1.797	1.0	0.909	0.909
Voltage (V)	1.1	0.8	0.67	0.75
Area (mm ²)	3.1	1.8	0.153	0.57
ResNet-50 Cycles	8.3M	4.4M	x	x
ResNet-50 ms	4.64	4.40	x	x
BERT-Base Cycles	x	x	10.4M	12.2M
BERT-Base ms	x	x	11.4	13.4
MAC Utilization	48%	95%	98%	98%

Table 7. Comparison of Voyager with Simba and 4-bit VSQ.

rapid design-space exploration, including the ability to evaluate quantized models on full datasets for precise accuracy estimation. Voyager-generated designs achieve consistently high utilization and performance across diverse models and at varying scales, significantly outperforming prior work. Compared to prior generators, Voyager-generated designs achieve up to 56% lower area and 61% lower runtime at iso-hardware configurations. Against hand-optimized accelerators, Voyager achieves up to 48% lower runtime, while requiring significantly less manual effort. We show that high performance DNN accelerators can be achieved without the extensive manual design and mapping effort that has been traditionally required.

8 Acknowledgments

This work was supported by funding from DARPA Young Faculty Award, COCOSYS SRC JUMP 2.0 Center, Amazon, Apple, Siemens and PORTAL Affiliates Program.

References

- [1] Alon Amid, David Biancolin, Abraham Gonzalez, Daniel Grubb, Sagar Karandikar, Harrison Liew, Albert Magyar, Howard Mao, Albert Ou, Nathan Pemberton, Paul Rigge, Colin Schmidt, John Wright, Jerry Zhao, Yakun Sophia Shao, Krste Asanović, and Borivoje Nikolić. Chipyard: Integrated design, simulation, and implementation framework for custom SoCs. *IEEE Micro*, 40(4):10–21, July 2020.
- [2] Jason Ansel, Edward Yang, Horace He, Natalia Gimelshein, Animesh Jain, Michael Voznesensky, Bin Bao, Peter Bell, David Berard, Evgeni Burovski, Geeta Chauhan, Anjali Chourdia, Will Constable, Alban Desmaison, Zachary DeVito, Elias Ellison, Will Feng, Jiong Gong, Michael Gschwind, Brian Hirsh, Sherlock Huang, Kshiteej Kalam-barkar, Laurent Kirsch, Michael Lazos, Mario Lezcano, Yanbo Liang, Jason Liang, Yinghai Lu, C. K. Luk, Bert Maher, Yunjie Pan, Christian Puhres, Matthias Reso, Mark Saroufim, Marcos Yukio Siraichi, Helen Suk, Shunting Zhang, Michael Suo, Phil Tillet, Xu Zhao, Eikan Wang, Keren Zhou, Richard Zou, Xiaodong Wang, Ajit Mathews, William Wen, Gregory Chanan, Peng Wu, and Soumith Chintala. PyTorch 2: Faster machine learning through dynamic python bytecode transformation and graph compilation. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages*

- and Operating Systems, Volume 2, ASPLOS '24, page 929–947, New York, NY, USA, 2024. Association for Computing Machinery.
- [3] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient finetuning of quantized LLMs. *Advances in Neural Information Processing Systems*, 36:10088–10115, 2023.
 - [4] Hasan Genc, Seah Kim, Alon Amid, Ameer Haj-Ali, Vighnesh Iyer, Pranav Prakash, Jerry Zhao, Daniel Grubb, Harrison Liew, Howard Mao, Albert Ou, Colin Schmidt, Samuel Steffl, John Wright, Ion Stoica, Jonathan Ragan-Kelley, Krste Asanovic, Borivoje Nikolic, and Yakun Sophia Shao. Gemmini: Enabling systematic deep-learning architecture evaluation via full-stack integration. In *Proceedings of the 58th Annual Design Automation Conference (DAC)*, 2021.
 - [5] Soroush Ghodrati, Sean Kinzer, Hanyang Xu, Rohan Mahapatra, Yoonsung Kim, Byung Hoon Ahn, Dong Kai Wang, Lavanya Karthikeyan, Amir Yazdanbakhsh, Jongse Park, Nam Sung Kim, and Hadi Esmaeilzadeh. Tandem processor: Grappling with emerging operators in neural networks. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2, ASPLOS '24*, page 1165–1182, New York, NY, USA, 2024. Association for Computing Machinery.
 - [6] John L Gustafson and Isaac T Yonemoto. Beating floating point at its own game: Posit arithmetic. *Supercomputing Frontiers and Innovations*, 4(2):71–86, June 2017.
 - [7] Ben Keller, Rangharajan Venkatesan, Steve Dai, Stephen G. Tell, Brian Zimmer, Charbel Sakr, William J. Dally, C. Thomas Gray, and Brucek Khailany. A 95.6-TOPS/W deep learning inference accelerator with per-vector scaled 4-bit quantization in 5 nm. *IEEE Journal of Solid-State Circuits*, 58(4):1129–1141, 2023.
 - [8] Hyoukjun Kwon, Prasantha Chatarasi, Vivek Sarkar, Tushar Krishna, Michael Pellauer, and Angshuman Parashar. MAESTRO: A data-centric approach to understand reuse, performance, and hardware cost of DNN mappings. *IEEE Micro*, 40(3):20–29, 2020.
 - [9] Hyoukjun Kwon, Ananda Samajdar, and Tushar Krishna. MAERI: Enabling flexible dataflow mapping over DNN accelerators via reconfigurable interconnects. *SIGPLAN Not.*, 53(2):461–475, March 2018.
 - [10] Linyan Mei, Pouya Houshmand, Vikram Jain, Sebastian Giraldo, and Marian Verhelst. ZigZag: Enlarging joint architecture-mapping design space exploration for DNN accelerators. *IEEE Transactions on Computers*, 70(8):1160–1174, 2021.
 - [11] Angshuman Parashar, Priyanka Raina, Yakun Sophia Shao, Yu-Hsin Chen, Victor A. Ying, Anurag Mukkara, Rangharajan Venkatesan, Brucek Khailany, Stephen W. Keckler, and Joel Emer. Timeloop: A systematic approach to DNN accelerator evaluation. In *2019 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, pages 304–315, 2019.
 - [12] Bitu Darvish Rouhani, Ritchie Zhao, Ankit More, Mathew Hall, Alireza Khodamoradi, Summer Deng, Dhruv Choudhary, Marius Cornea, Eric Dellinger, Kristof Denolf, Stosic Dusan, Venmugil Elango, Maximilian Golub, Alexander Heinecke, Phil James-Roxby, Dharmesh Jani, Gaurav Kolhe, Martin Langhammer, Ada Li, Levi Melnick, Maral Mesmakhosroshahi, Andres Rodriguez, Michael Schulte, Rasoul Shafipour, Lei Shao, Michael Siu, Pradeep Dubey, Paulius Micikevicius, Maxim Naumov, Colin Verrilli, Ralph Wittig, Doug Burger, and Eric Chung. Microscaling data formats for deep learning, 2023.
 - [13] Frans Sijstermans. The NVIDIA deep learning accelerator. In *2018 IEEE Hot Chips 30 Symposium (HCS)*, Cupertino, CA, USA, August 19–21, 2018. IEEE, 2018.
 - [14] TorchVision maintainers and contributors. TorchVision: PyTorch’s computer vision library. <https://github.com/pytorch/vision>, 2016.
 - [15] Rangharajan Venkatesan, Yakun Sophia Shao, Miaorong Wang, Jason Clemons, Steve Dai, Matthew Fojtik, Ben Keller, Alicia Klinefelter, Nathaniel Pinckney, Priyanka Raina, Yanqing Zhang, Brian Zimmer, William J. Dally, Joel Emer, Stephen W. Keckler, and Brucek Khailany. MAGNet: A modular accelerator generator for neural networks. In *2019 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, pages 1–8, 2019.
 - [16] Rangharajan Venkatesan, Yakun Sophia Shao, Brian Zimmer, Jason Clemons, Matthew Fojtik, Nan Jiang, Ben Keller, Alicia Klinefelter, Nathaniel Ross Pinckney, Priyanka Raina, Stephen G. Tell, Yanqing Zhang, William J. Dally, Joel S. Emer, C. Thomas Gray, Stephen W. Keckler, and Brucek Khailany. A 0.11 pJ/op, 0.32–128 TOPS, scalable multi-chip-module-based deep neural network accelerator designed with a high-productivity VLSI methodology. In *2019 IEEE Hot Chips 31 Symposium (HCS)*, Cupertino, CA, USA, August 18–20, 2019, pages 1–24. IEEE, 2019.
 - [17] Ross Wightman. PyTorch image models. <https://github.com/huggingface/pytorch-image-models>, 2019.
 - [18] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, Mariama Drame, Quentin Lhoest, and Alexander M. Rush. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 38–45, Online, October 2020. Association for Computational Linguistics.
 - [19] Xuan Yang, Mingyu Gao, Qiaoyi Liu, Jeff Setter, Jing Pu, Ankita Nayak, Steven Bell, Kaidi Cao, Heonjae Ha, Priyanka Raina, Christos Kozyrakis, and Mark Horowitz. Interstellar: Using Halide’s scheduling language to analyze DNN accelerators. In *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems, ASPLOS '20*, page 369–383, New York, NY, USA, 2020. Association for Computing Machinery.
 - [20] Xiaofan Zhang, Junsong Wang, Chao Zhu, Yonghua Lin, Jinjun Xiong, Wen-mei Hwu, and Deming Chen. DNNBuilder: An automated tool for building high-performance DNN hardware accelerators for FPGAs. In *2018 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, pages 1–8, 2018.
 - [21] Brian Zimmer, Rangharajan Venkatesan, Yakun Sophia Shao, Jason Clemons, Matthew Fojtik, Nan Jiang, Ben Keller, Alicia Klinefelter, Nathaniel Pinckney, Priyanka Raina, Stephen G. Tell, Yanqing Zhang, William J. Dally, Joel S. Emer, C. Thomas Gray, Stephen W. Keckler, and Brucek Khailany. A 0.11 pJ/op, 0.32–128 TOPS, scalable multi-chip-module-based deep neural network accelerator with ground-reference signaling in 16nm. In *2019 Symposium on VLSI Circuits*, pages C300–C301, 2019.
 - [22] Brian Zimmer, Rangharajan Venkatesan, Yakun Sophia Shao, Jason Clemons, Matthew Fojtik, Nan Jiang, Ben Keller, Alicia Klinefelter, Nathaniel Pinckney, Priyanka Raina, Stephen G. Tell, Yanqing Zhang, William J. Dally, Joel S. Emer, C. Thomas Gray, Stephen W. Keckler, and Brucek Khailany. A 0.32–128 TOPS, scalable multi-chip-module-based deep neural network inference accelerator with ground-referenced signaling in 16 nm. *IEEE Journal of Solid-State Circuits*, 55(4):920–932, 2020.

A Complete Runtime Results

Table 8, Table 9, Table 10, Table 11, and Table 12 show detailed runtime results per layer for each reported model for 16×16 and 32×32 designs, to allow easy comparisons with related work. The total MAC utilization is calculated by summing ideal cycles for the matrix operations and summing achieved cycles for the matrix operations, and dividing ideal by achieved.

Table 8. ResNet-18 performance and MAC utilization per layer.

Layer	Count	Unit	16×16			32×32		
			Ideal Cycles	Achieved Cycles	MAC Utilization	Ideal Cycles	Achieved Cycles	MAC Utilization
conv1_fused 224×224×3 * 7×7×3×64	1	matrix	460992	956615	48.19%	115248	246291	46.79%
max_pool2d_default 112×112×64	1	vector	–	223641	–	–	111859	–
quantize_default_1 56×56×64	1	vector	–	25163	–	–	12618	–
layer1_0_conv1_fused 56×56×64 * 3×3×64×64	2	matrix	451584	451850	99.94%	112896	113366	99.59%
layer1_0_conv2_fused 56×56×64 * 3×3×64×64	2	matrix	451584	451850	99.94%	112896	113366	99.59%
layer2_0_conv1_fused 56×56×64 * 3×3×64×128	1	matrix	225792	226378	99.74%	56448	57518	98.14%
layer2_0_conv2_fused 28×28×128 * 3×3×128×128	1	matrix	451584	451870	99.94%	112896	113355	99.59%
layer2_0_downsample_0_fused 56×56×64 * 1×1×64×128	1	matrix	25088	25222	99.47%	6272	6491	96.62%
layer2_1_conv1_fused 28×28×128 * 3×3×128×128	1	matrix	451584	451870	99.94%	112896	113355	99.59%
layer2_1_conv2_fused 28×28×128 * 3×3×128×128	1	matrix	451584	451870	99.94%	112896	113355	99.59%
layer3_0_conv1_fused 28×28×128 * 3×3×128×256	1	matrix	225792	226378	99.74%	56448	57518	98.14%
layer3_0_conv2_fused 14×14×256 * 3×3×256×256	1	matrix	451584	451835	99.94%	112896	113355	99.59%
layer3_0_downsample_0_fused 28×28×128 * 1×1×128×256	1	matrix	25088	25222	99.47%	6272	6491	96.62%
layer3_1_conv1_fused 14×14×256 * 3×3×256×256	1	matrix	451584	451835	99.94%	112896	113355	99.59%
layer3_1_conv2_fused 14×14×256 * 3×3×256×256	1	matrix	451584	451835	99.94%	112896	113355	99.59%
layer4_0_conv1_fused 14×14×256 * 3×3×256×512	1	matrix	225792	226154	99.84%	56448	56907	99.19%
layer4_0_conv2_fused 7×7×512 * 3×3×512×512	1	matrix	451584	451835	99.94%	112896	113355	99.59%
layer4_0_downsample_0_fused 14×14×256 * 1×1×256×512	1	matrix	25088	25243	99.39%	6272	6491	96.62%
layer4_1_conv1_fused 7×7×512 * 3×3×512×512	1	matrix	451584	451835	99.94%	112896	113809	99.19%
layer4_1_conv2_fused 7×7×512 * 3×3×512×512	1	matrix	451584	452597	99.78%	112896	113809	99.19%
adaptive_avg_pool2d_default 7×7×512	1	vector	–	3212	–	–	1643	–
fc 1×512 * 1000×512	1	vector	–	66038	–	–	33581	–
Total			–	7904048	93.0%	–	2071975	91.9%

Table 9. ResNet-50 performance and MAC utilization per layer.

Layer	Count	Unit	16×16			32×32		
			Ideal Cycles	Achieved Cycles	MAC Utilization	Ideal Cycles	Achieved Cycles	MAC Utilization
quantize_default_1 56×56×64	1	vector	–	25163	–	–	12618	–
conv1_fused 224×224×3 * 7×7×3×64	1	matrix	460992	956615	48.19%	115248	246291	46.79%
max_pool2d_default 112×112×64	1	vector	–	223641	–	–	111859	–
layer1_0_conv1_fused 56×56×64 * 1×1×64×64	1	matrix	50176	50310	99.73%	12544	12763	98.28%
layer1_0_conv2_fused 56×56×64 * 3×3×64×64	3	matrix	451584	451850	99.94%	112896	113366	99.59%
layer1_0_conv3_fused 56×56×64 * 1×1×64×256	1	matrix	200704	200838	99.93%	50176	50395	99.56%
layer1_0_downsample_0_fused 56×56×64 * 1×1×64×256	1	matrix	200704	200838	99.93%	50176	50395	99.56%
layer1_1_conv1_fused 56×56×256 * 1×1×256×64	2	matrix	200704	200838	99.93%	50176	50395	99.56%
layer1_1_conv2_fused 56×56×64 * 3×3×64×64	2	matrix	451584	451870	99.94%	112896	113355	99.59%
layer1_1_conv3_fused 56×56×64 * 1×1×64×256	3	matrix	200704	200838	99.93%	50176	50395	99.56%
layer2_0_conv1_fused 56×56×256 * 1×1×256×128	1	matrix	401408	401542	99.97%	100352	100571	99.78%
layer2_0_conv2_fused 28×28×128 * 3×3×128×128	1	matrix	451584	452170	99.87%	112896	113966	99.06%
layer2_0_conv3_fused 28×28×128 * 1×1×128×512	1	matrix	200704	200838	99.93%	50176	50395	99.56%
layer2_0_downsample_0_fused 56×56×256 * 1×1×256×512	1	matrix	401408	401542	99.97%	100352	100571	99.78%
layer2_1_conv1_fused 28×28×512 * 1×1×512×128	3	matrix	200704	200838	99.93%	50176	50395	99.56%
layer2_1_conv2_fused 28×28×128 * 3×3×128×128	3	matrix	451584	451870	99.94%	112896	113355	99.59%
layer2_1_conv3_fused 28×28×128 * 1×1×128×512	1	matrix	200704	200838	99.93%	50176	50395	99.56%
layer3_0_conv1_fused 28×28×512 * 1×1×512×256	1	matrix	401408	401542	99.97%	100352	100571	99.78%
layer3_0_conv2_fused 14×14×256 * 3×3×256×256	1	matrix	451584	452170	99.87%	112896	113966	99.06%
layer3_0_conv3_fused 14×14×256 * 1×1×256×1024	1	matrix	200704	200838	99.93%	50176	50395	99.56%
layer3_0_downsample_0_fused 28×28×512 * 1×1×512×1024	1	matrix	401408	401542	99.97%	100352	100571	99.78%
layer3_1_conv1_fused 14×14×1024 * 1×1×1024×256	5	matrix	200704	200838	99.93%	50176	50395	99.56%
layer3_1_conv2_fused 14×14×256 * 3×3×256×256	5	matrix	451584	451835	99.94%	112896	113355	99.59%

Continued on next page...

Table 9. ResNet-50 performance and MAC utilization per layer.

Layer	Count	Unit	16×16			32×32		
			Ideal Cycles	Achieved Cycles	MAC Utilization	Ideal Cycles	Achieved Cycles	MAC Utilization
layer3_1_conv3_fused 14×14×256 * 1×1×256×1024	1	matrix	200704	200838	99.93%	50176	50395	99.56%
layer4_0_conv1_fused 14×14×1024 * 1×1×1024×512	1	matrix	401408	401542	99.97%	100352	100571	99.78%
layer4_0_conv2_fused 7×7×512 * 3×3×512×512	1	matrix	451584	451946	99.92%	112896	113355	99.59%
layer4_0_conv3_fused 7×7×512 * 1×1×512×2048	1	matrix	200704	200859	99.92%	50176	50395	99.56%
layer4_0_downsample_0_fused 14×14×1024 * 1×1×1024×2048	1	matrix	401408	401563	99.96%	100352	100571	99.78%
layer4_1_conv1_fused 7×7×2048 * 1×1×2048×512	2	matrix	200704	200859	99.92%	50176	50395	99.56%
layer4_1_conv2_fused 7×7×512 * 3×3×512×512	2	matrix	451584	451835	99.94%	112896	113355	99.59%
layer4_1_conv3_fused 7×7×512 * 1×1×512×2048	1	matrix	200704	200859	99.92%	50176	50395	99.56%
layer4_2_conv3_fused 7×7×512 * 1×1×512×2048	1	matrix	200704	205493	97.67%	50176	52145	96.22%
adaptive_avg_pool2d_default 7×7×2048	1	vector	–	12620	–	–	6347	–
fc 1×2048 * 1000×2048	1	vector	–	259574	–	–	131885	–
Total			–	16896078	96.2%	–	4378064	94.9%

Table 10. ViT performance and MAC utilization per layer.

Layer	Count	Unit	16×16			32×32		
			Ideal Cycles	Achieved Cycles	MAC Utilization	Ideal Cycles	Achieved Cycles	MAC Utilization
add 197×768	1	vector	–	18989	–	–	9533	–
add_scalar 1×768	1	vector	–	173	–	–	125	–
classifier 1×768 * 768×1000	1	vector	–	98294	–	–	49965	–
layer_norm_24 224×768	1	vector	–	86336	–	–	43326	–
layer_norm_fused 224×768	24	vector	–	86334	–	–	43323	–
matmul_24_1_fused 224×64 * 64×256	144	matrix	12544	14500	86.51%	3136	4439	70.65%
matmul_36_1_fused 224×224 * 224×64	144	matrix	12544	12678	99.09%	3136	3362	93.28%
quantize_default_7 224×768	23	vector	–	21579	–	–	10826	–
Continued on next page...								

Table 10. ViT performance and MAC utilization per layer.

Layer	Count	Unit	16×16			32×32		
			Ideal Cycles	Achieved Cycles	MAC Utilization	Ideal Cycles	Achieved Cycles	MAC Utilization
softmax_12_fused 224×256	144	vector	–	19057	–	–	9646	–
patch_embeddings_projection 224×224×3 * 16×16×3×768	1	matrix	451584	603197	74.87%	112896	172323	65.51%
attention_key_fused 224×768 * 768×768	36	matrix	516096	516230	99.97%	129024	129250	99.83%
attention_output_dense_fused_0 12×224×64 * 768×768	1	matrix	516096	525447	98.22%	129024	134528	95.91%
intermediate_dense_fused 224×768 * 768×3072	12	matrix	2064384	2064930	99.97%	516096	519608	99.32%
output_dense_fused 224×3072 * 3072×768	12	matrix	2064384	2064930	99.97%	516096	519608	99.32%
attention_output_dense_fused_1 12×224×64 * 768×768	11	matrix	516096	516641	99.90%	129024	132536	97.35%
Total			–	84369325	99.3%	–	22741874	98.0%

Table 11. BERT-Base performance and MAC utilization per layer.

Layer	Count	Unit	16×16			32×32		
			Ideal Cycles	Achieved Cycles	MAC Utilization	Ideal Cycles	Achieved Cycles	MAC Utilization
attention_output_dense_fused 128×768 * 768×768	12	matrix	294912	298537	98.79%	73728	76485	96.40%
attention_self_key_fused 128×768 * 768×768	36	matrix	294912	295053	99.95%	73728	73965	99.68%
intermediate_dense_fused 128×768 * 768×3072	12	matrix	1179648	1179790	99.99%	294912	295149	99.92%
output_dense_fused 128×3072 * 3072×768	12	matrix	1179648	1180393	99.94%	294912	297092	99.26%
classifier 1×768 * 768×2	1	vector	–	1635	–	–	1634	–
layer_norm_23 128×768	1	vector	–	49464	–	3072	24886	–
layer_norm_fused 128×768	23	vector	–	49462	–	3072	24883	–
matmul_24_fused 128×64 * 64×128	144	matrix	4096	5135	79.77%	1024	1778	57.57%
matmul_36_fused 128×128 * 128×64	144	matrix	4096	4237	96.67%	1024	1261	81.18%
pooler_dense 1×768 * 768×768	1	matrix	36864	74955	49.17%	18432	37491	49.15%
quantize_default 128×768	1	vector	–	12361	–	–	6216	–
softmax_12_fused 128×128	144	vector	–	6379	–	–	3304	–
Continued on next page...								

Table 11. BERT-Base performance and MAC utilization per layer.

Layer	Count	Unit	16×16			32×32		
			Ideal Cycles	Achieved Cycles	MAC Utilization	Ideal Cycles	Achieved Cycles	MAC Utilization
tanh 1×768	1	vector	–	171	–	–	123	–
Total			–	46070904	99.4%	–	12243503	97.9%

Table 12. MobileBERT-TINY performance and MAC utilization per layer.

Layer	Count	Unit	16×16			32×32		
			Ideal Cycles	Achieved Cycles	MAC Utilization	Ideal Cycles	Achieved Cycles	MAC Utilization
add_10 128×512	21	vector	–	8267	–	–	4171	–
add_4_fused 128×128	63	vector	–	2121	–	–	1096	–
classifier 1×512 * 512×2	21	vector	–	1123	–	–	1122	–
matmul_2_fused 128×32 * 32×128	84	matrix	2048	3094	66.18%	512	1262	40.57%
matmul_6_fused 128×128 * 128×32	84	matrix	2048	2189	93.56%	512	748	68.45%
attention_output_dense_fused 4×128×32 * 128×128	21	matrix	8192	9294	88.12%	2048	2791	73.38%
attention_self_query_fused 128×128 * 128×128	42	matrix	8192	8333	98.31%	2048	2285	89.60%
attention_self_value_fused 128×512 * 512×128	21	matrix	32768	32908	99.57%	8192	8428	97.19%
bottleneck_input_dense_fused 128×512 * 512×128	42	matrix	32768	33802	96.94%	8192	8926	91.78%
ffn_intermediate_dense_fused 128×128 * 128×512	42	matrix	32768	32909	99.57%	8192	8429	97.18%
ffn_output_dense_fused 128×512 * 512×128	42	matrix	32768	33805	96.93%	8192	8930	91.77%
output_bottleneck_dense_fused 128×128 * 128×512	21	matrix	32768	36750	89.16%	8192	10459	78.32%
quantize_default 128×512	21	vector	–	8265	–	–	4168	–
softmax_1_fused 128×128	84	vector	–	6379	–	–	3304	–
Total			–	7713636	95.1%	–	2369283	86.2%

Table 13. LLaMA performance and MAC utilization per layer.

Layer	Count	Unit	32×32		
			Ideal Cycles	Achieved Cycles	MAC Utilization
add_39_fused 1×8×512×64	128	vector	–	16,456	–
Continued on next page...					

Table 13. LLaMA performance and MAC utilization per layer.

Layer	Count	Unit	32×32		
			Ideal Cycles	Achieved Cycles	MAC Utilization
layernorm_default_fused 1×32×64×512	32	vector	–	262,451	–
matmul_2_fused 512×512 * 512×64	512	matrix	16384	16754	97.79%
matmul_34_fused 512×64 * 64×512	512	matrix	16384	16620	98.58%
model_layers_0_mlp_down_proj_fused 512×8192 * 8192×2048	16	matrix	8,388,608	8,390,496	99.98%
model_layers_0_mlp_gate_proj_fused 512×2048 * 2048×8192	16	matrix	16,777,216	16,780,160	99.98%
model_layers_0_mlp_up_proj_fused 512×2048 * 2048×8192	16	matrix	8,388,608	8,391,520	99.79%
model_layers_0_self_attn_k_proj_fused 512×2048 * 2048×512	32	matrix	524,288	524,656	99.93%
model_layers_0_self_attn_o_proj_fused 512×2048 * 2048×2048	16	matrix	2,097,152	2,097,520	99.98%
model_layers_0_self_attn_q_proj_fused 512×2048 * 2048×2048	16	matrix	2,097,152	2,097,520	99.98%
mul_2_fused 512×2048 * 2048×2048	16	vector	–	65608	–
mul_4_fused 512×2048 * 2048×2048	16	vector	–	16456	–
permute_default_1_fused 4×1×8×512×64	16	vector	–	16459	–
permute_default_fused 512×2048 * 2048×2048	16	vector	–	65608	–
quantize_default 32×64×512	32	vector	–	65608	–
slice_1 32×512×64 -> 32×64×512	16	vector	–	32843	–
slice_2_fused 32×512×64 -> 512×32×64	16	vector	–	32843	–
slice_3 32×512×64 -> 32×64×512	16	vector	–	8267	–
slice_4_fused 32×512×64 -> 512×32×64	16	vector	–	8267	–
softmax_1_fused 512×512	512	vector	–	49384	–
lm_head 512×2048 * 2048×126976	1	matrix	8,126,464	8,127,936	99.98%
Total			–	687,950,352	99.8%

Table 14. MobileNetV2 performance and MAC utilization per layer.

Layer	Count	Unit	16×16			32×32		
			Ideal Cycles	Achieved Cycles	MAC Utilization	Ideal Cycles	Achieved Cycles	MAC Utilization
features_0_0_fused 224×224×3 * 3×3×3×32	1	matrix	42336	466179	9.08%	10584	117505	9.00%
features_1_conv_0_0_fused 112×112×32 * 3×3×1×32	1	dwc	25088	26794	93.63%	12544	13460	93.19%
features_1_conv_1_fused 112×112×16 * 1×1×32×16	1	matrix	25088	25224	99.46%	6272	12765	49.13%
features_2_conv_0_0_fused 112×112×16 * 1×1×16×96	1	matrix	75264	75400	99.82%	18816	37853	49.71%
features_2_conv_1_0_fused 112×112×96 * 3×3×1×96	1	dwc	18816	80155	23.47%	9408	40153	23.43%
features_2_conv_2_fused 56×56×96 * 1×1×96×24	1	matrix	28224	37768	74.73%	7056	9629	73.28%
features_3_conv_0_0_fused 56×56×24 * 1×1×24×144	2	matrix	42336	56584	74.82%	10584	15901	66.56%
features_3_conv_1_0_fused 56×56×144 * 3×3×1×144	1	dwc	28224	31018	90.99%	14112	17354	81.32%
features_3_conv_2_fused 56×56×144 * 1×1×144×24	1	matrix	42336	56584	74.82%	10584	15901	66.56%
features_4_conv_1_0_fused 56×56×144 * 3×3×1×144	1	dwc	7056	30997	22.76%	3528	17333	20.35%
features_4_conv_2_fused 28×28×144 * 1×1×144×32	1	matrix	14112	14248	99.05%	3528	4141	85.20%
features_5_conv_0_0_fused 28×28×32 * 1×1×32×192	3	matrix	18816	18952	99.28%	4704	4925	95.51%
features_5_conv_1_0_fused 28×28×192 * 3×3×1×192	2	dwc	9408	10726	87.71%	4704	5506	85.43%
features_5_conv_2_fused 28×28×192 * 1×1×192×32	2	matrix	18816	18952	99.28%	4704	4925	95.51%
features_7_conv_1_0_fused 28×28×192 * 3×3×1×192	1	dwc	2352	10695	21.99%	1176	5475	21.48%
features_7_conv_2_fused 14×14×192 * 1×1×192×64	1	matrix	9408	9544	98.58%	2352	2573	91.41%
features_8_conv_0_0_fused 14×14×64 * 1×1×64×384	4	matrix	18816	18952	99.28%	4704	4925	95.51%
features_8_conv_1_0_fused 14×14×384 * 3×3×1×384	4	dwc	4704	6238	75.41%	2352	3358	70.04%
features_8_conv_2_fused 14×14×384 * 1×1×384×64	3	matrix	18816	18952	99.28%	4704	4925	95.51%
features_11_conv_2_fused 14×14×384 * 1×1×384×96	1	matrix	28224	28360	99.52%	7056	7277	96.96%
features_12_conv_0_0_fused 14×14×96 * 1×1×96×576	3	matrix	42336	42472	99.68%	10584	10805	97.95%
features_12_conv_1_0_fused 14×14×576 * 3×3×1×576	2	dwc	7056	9310	75.79%	3528	4990	70.70%
features_12_conv_2_fused 14×14×576 * 1×1×576×96	2	matrix	42336	42472	99.68%	10584	10805	97.95%

Continued on next page...

Table 14. MobileNetV2 performance and MAC utilization per layer.

Layer	Count	Unit	16×16			32×32		
			Ideal Cycles	Achieved Cycles	MAC Utilization	Ideal Cycles	Achieved Cycles	MAC Utilization
features_14_conv_1_0_fused 14×14×576 * 3×3×1×576	1	dwc	1764	9293	18.98%	882	4973	17.73%
features_14_conv_2_fused 7×7×576 * 1×1×576×160	1	matrix	17640	17797	99.12%	4410	4631	95.23%
features_15_conv_0_0_fused 7×7×160 * 1×1×160×960	3	matrix	29400	29557	99.47%	7350	7571	97.08%
features_15_conv_1_0_fused 7×7×960 * 3×3×1×960	3	dwc	2940	5374	54.71%	1470	3214	45.74%
features_15_conv_2_fused 7×7×960 * 1×1×960×160	2	matrix	29400	29557	99.47%	7350	7571	97.08%
features_17_conv_2_fused 7×7×960 * 1×1×960×320	1	matrix	58800	58957	99.73%	14700	14921	98.52%
features_18_0_fused 7×7×320 * 1×1×320×1280	1	matrix	78400	78659	99.67%	19600	19923	98.38%
adaptive_avg_pool2d_default 7×7×1280	1	vector	–	7914	–	–	3993	–
classifier_1 1280 * 1280×1000	1	vector	–	162867	–	–	82731	–
Total			–	2010336	83.7%	–	659439	78.0%